

Day 3 — ORDER BY, LIMIT, and DISTINCT

Today you'll learn how to **control the order and number of rows returned by SQL** and how to remove duplicate values.

These are simple commands, but they're heavily used in APIs, dashboards, analytics, and RAG applications.

Rule: You write the SQL. I will **not provide answers before you attempt the exercises.**

1. Today's Goals

By the end of Day 3, you should understand:

- ORDER BY
 - Ascending vs descending order
 - Ordering by multiple columns
 - Ordering by expressions
 - LIMIT
 - Why ordering matters with LIMIT
 - DISTINCT
 - DISTINCT across multiple columns
 - Combining WHERE + ORDER BY + LIMIT
 - Practical pagination concepts
 - Common SQL mistakes and debugging
-

2. Dataset

We'll continue with our RAG database.

documents

id	user_id	title	file_type	page_count	status
101	1	RAG Fundamentals	pdf	20	processed
102	1	PostgreSQL Guide	pdf	45	processed
103	2	Python Basics	pdf	30	processing
104	3	FastAPI Tutorial	md	15	failed
105	4	Vector Databases	pdf	50	processed
106	5	Docker Guide	pdf	25	processing

users

id	name	country	account_status
1	Aarav	India	active
2	Emma	USA	active

id	name	country	account_status
----	------	---------	----------------

3	Liam	UK	inactive
4	Sofia	India	active
5	Noah	USA	suspended

3. ORDER BY

Suppose we run:

```
SELECT
```

```
title,
```

```
page_count
```

```
FROM documents
```

```
ORDER BY page_count;
```

ORDER BY tells PostgreSQL:

Sort the result according to this column.

By default, ascending order is used.

Conceptually:

15

20

25

30

45

50

4. ASC

You can explicitly specify ascending:

```
SELECT
```

```
title,
```

```
page_count
```

```
FROM documents
```

```
ORDER BY page_count ASC;
```

ASC means:

smallest → largest

For text, generally:

A → Z

5. DESC

Descending:

SELECT

title,

page_count

FROM documents

ORDER BY page_count DESC;

Means:

largest → smallest

So:

50

45

30

25

20

15

Easy memory trick

ASC → up

DESC → down

6. Why Sorting Matters in RAG Systems

Imagine your RAG backend stores document processing information.

An admin asks:

Show me the largest documents first.

You need:

ORDER BY page_count DESC

Another requirement:

Show the newest documents first.

You might eventually use:

ORDER BY created_at DESC

Another:

Show failed documents first.

You can sort by a status-related expression or other fields.

Ordering is therefore not just a presentation feature—it can be part of your application's business logic.

7. Ordering by Multiple Columns

Suppose two documents have the same `page_count`.

You can define a secondary sort:

```
SELECT
title,
page_count
  FROM documents
  ORDER BY
page_count DESC,
title ASC;
```

PostgreSQL does this conceptually:

1. Sort by `page_count DESC`

2. If `page_count` is equal,

sort those rows by title ASC

For example:

```
50 Vector Databases
45 PostgreSQL Guide
30 Python Basics
25 Docker Guide
20 RAG Fundamentals
15 FastAPI Tutorial
```

If two documents both had 30 pages, their titles would determine their order.

8. Different Direction for Each Column

You can do:

```
SELECT
title,
page_count
  FROM documents
  ORDER BY
page_count DESC,
title ASC;
```

The first column is descending.

The second is ascending.

You don't have to use the same direction for everything.

9. LIMIT

Now suppose there are thousands of documents.

You only want the first five:

```
SELECT
```

```
id,
```

```
title
```

```
    FROM documents
```

```
LIMIT 5;
```

LIMIT means:

Return at most this many rows.

10. ORDER BY + LIMIT

This is much more useful.

Suppose you want:

The 3 largest documents.

```
SELECT
```

```
id,
```

```
title,
```

```
page_count
```

```
    FROM documents
```

```
    ORDER BY page_count DESC
```

```
LIMIT 3;
```

The conceptual process is:

documents

↓

sort by page_count DESC

↓

take first 3

↓

return

This pattern is extremely common.

11. LIMIT Without ORDER BY

Consider:

```
SELECT *  
FROM documents
```

LIMIT 3;

You should **not interpret this as:**

Give me the first three documents.

SQL tables don't inherently guarantee a meaningful row order.

Without ORDER BY, you should not rely on which three rows PostgreSQL returns.

If you care which rows you receive, explicitly define the ordering.

For example:

```
SELECT *  
FROM documents  
ORDER BY id
```

LIMIT 3;

Now you've defined what "first" means.

12. DISTINCT

Suppose:

```
SELECT country  
FROM users;
```

could produce:

India

USA

UK

India

USA

There are duplicates.

Use:

```
SELECT DISTINCT country  
FROM users;
```

Result:

India

USA

UK

DISTINCT removes duplicate result rows.

13. DISTINCT Is About the Result

Important:

```
SELECT DISTINCT country  
FROM users;
```

does **not** modify the table.

It doesn't delete duplicate users.

It only says:

Return unique country values in this query result.

14. DISTINCT With Multiple Columns

Consider:

```
SELECT DISTINCT  
country,  
account_status  
FROM users;
```

Here PostgreSQL considers the **combination**:

country + account_status

to determine uniqueness.

For example:

India + active

USA + active

UK + inactive

USA + suspended

If another row has:

India + active

that combination is a duplicate.

But:

India + suspended

would be a different combination.

15. DISTINCT + ORDER BY

You can combine them:

```
SELECT DISTINCT country  
FROM users
```

ORDER BY country ASC;

Now you get unique countries in alphabetical order.

16. WHERE + ORDER BY + LIMIT

This combination is extremely important.

Suppose the RAG admin asks:

Show the 2 largest processed PDF documents.

You need three operations:

Step 1 — Filter

processed

PDF

Step 2 — Sort

largest first

Step 3 — Limit

2 rows

The SQL structure is:

SELECT

id,

title,

page_count

FROM documents

WHERE file_type = 'pdf'

AND status = 'processed'

ORDER BY page_count DESC

LIMIT 2;

Notice the logical structure:

SELECT

FROM

WHERE

ORDER BY

LIMIT

We'll build much more complicated versions of this later.

17. SQL Query Clause Order

A useful syntax pattern to memorize:

```
SELECT ...  
FROM ...  
WHERE ...  
ORDER BY ...
```

```
LIMIT ...;
```

For now, keep this exact structure in your head.

For example:

```
SELECT  
title,  
page_count  
  FROM documents  
 WHERE page_count > 20  
 ORDER BY page_count DESC  
LIMIT 3;
```

18. LIMIT in APIs

Imagine your FastAPI endpoint:

GET /documents

The database contains:

500,000 documents

Returning every row is usually a terrible API design.

Instead, you might retrieve:

20 documents

using:

LIMIT 20

Later, you'll learn proper pagination patterns.

19. A First Look at OFFSET

We're not studying pagination deeply today, but you should know this exists:

```
SELECT  
id,  
title  
  FROM documents  
 ORDER BY id  
LIMIT 10
```

OFFSET 20;

Conceptually:

skip 20 rows

then return 10 rows

So:

OFFSET 20

LIMIT 10

roughly corresponds to rows 21–30 of the ordered result.

Important

Don't confuse:

LIMIT

with:

OFFSET

LIMIT → how many to return.

OFFSET → how many to skip first.

We'll revisit pagination from an engineering perspective later.

20. Deterministic Ordering

Consider:

```
SELECT *
```

```
FROM documents
```

```
ORDER BY page_count DESC
```

LIMIT 3;

What happens if many documents have the same page_count?

The database has flexibility regarding their relative order.

For APIs, you often want a deterministic tie-breaker:

```
SELECT *
```

```
FROM documents
```

```
ORDER BY
```

```
page_count DESC,
```

```
id ASC
```

LIMIT 3;

Now if two documents have equal page counts, id determines the order.

This becomes important for stable pagination.

21. Debugging Challenge #1

A developer writes:

```
SELECT
title,
page_count
  FROM documents
LIMIT 3
  ORDER BY page_count DESC;
```

PostgreSQL rejects the query.

Your tasks:

1. What is wrong?
 2. Rewrite it correctly.
 3. Explain the correct clause order.
-

22. Debugging Challenge #2

A developer wants:

The 5 largest PDF documents.

They write:

```
SELECT
title,
page_count
  FROM documents
LIMIT 5
  WHERE file_type = 'pdf'
  ORDER BY page_count DESC;
```

Find all problems.

There are **multiple** issues.

23. Exercise Set A — ORDER BY

Q1

Return all documents sorted by page_count from smallest to largest.

Q2

Return all documents sorted by page_count from largest to smallest.

Q3

Return:

title

page_count

sorted by title alphabetically.

Q4

Sort documents by:

1. status ascending
2. page_count descending

Write one query.

Q5

Sort documents by page_count DESC, and use id ASC as the tie-breaker.

24. Exercise Set B — LIMIT

Q6

Return any 3 documents.

Q7

Return the 2 documents with the highest page counts.

Q8

Return the 3 smallest documents by page count.

Q9

Return the 2 largest **processed PDF documents**.

Return:

id

title

page_count

25. Exercise Set C — DISTINCT

Q10

Return all unique countries from users.

Q11

Return all unique file types from documents.

Q12

Return unique combinations of:

file_type

status

from documents.

Q13

Return unique countries sorted alphabetically.

26. Practical Engineering Problem

You're building a RAG admin dashboard.

The dashboard displays:

Top 3 largest successfully processed PDF documents

Requirements:

- Only PDFs
- Only processed
- Largest page count first
- If page counts are equal, smaller id first
- Return only 3 rows

Return:

document_id

title

pages

You must use:

- WHERE

- ORDER BY
- LIMIT
- column aliases

Write **one SQL query**.

27. Practical API Problem

Your FastAPI endpoint needs:

Return the newest 10 documents.

The table has:

id

title

created_at

Task

Write the SQL query.

Then answer:

Why would LIMIT 10 without ORDER BY created_at be wrong for this API requirement?

28. Production Debugging

A developer creates an endpoint:

GET /users?limit=10

Their SQL is:

SELECT

id,

name,

email

FROM users

LIMIT 10;

The product manager says:

"Every time I refresh the endpoint, I want the same first 10 users, ordered by user ID."

Your tasks:

1. What's missing?

2. Rewrite the query.

3. Why is deterministic ordering important?

29. Interview Questions

Answer these as if you're in a technical interview.

Q1

What's the difference between:

```
ORDER BY page_count ASC
```

and:

```
ORDER BY page_count DESC
```

Q2

Why should you generally use ORDER BY when using LIMIT if you care which rows are returned?

Q3

What does DISTINCT do?

Q4

What happens when you write:

```
SELECT DISTINCT country, account_status  
FROM users;
```

What exactly is considered a duplicate?

30. Explanation Challenge

Explain this query in your own words:

```
SELECT  
id AS document_id,  
title,  
page_count AS pages  
FROM documents  
WHERE file_type = 'pdf'  
AND status = 'processed'  
ORDER BY  
page_count DESC,  
id ASC  
LIMIT 3;
```

Don't explain each keyword individually.

Explain **what the query accomplishes from a business perspective**.

31. Think Like a Data Engineer

Answer this without writing SQL:

You have:

1,000,000 documents

and someone writes:

```
SELECT *
```

```
FROM documents;
```

Why might this be problematic in a production RAG system?

Think about:

- Network
 - Memory
 - API response size
 - Database workload
 - Large columns such as embeddings
 - Application performance
-

32. Day 3 Short Test

Q1

What does ORDER BY do?

Q2

What is the default ordering direction?

Q3

Write a query to sort documents from largest to smallest by page count.

Q4

What does LIMIT 5 mean?

Q5

Why can this be problematic?

```
SELECT *  
FROM documents
```

```
LIMIT 5;
```

Q6

What does DISTINCT do?

Q7

Write a query returning unique file_type values.

Q8

What does this mean?

```
ORDER BY page_count DESC, id ASC
```

Q9

Write a query to return the 3 largest processed documents.

Q10

Explain why this clause order is wrong:

```
SELECT *  
FROM documents
```

```
LIMIT 5
```

```
WHERE status = 'processed';
```
